Import Libraries

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as mtick

sns.set(style="whitegrid")
```

Load Datasets

```python
import os

BASE_DIR = os.path.join("..", "data")

trips = pd.read_csv(os.path.join(BASE_DIR, "raw_data", "Uber Trip Details.csv"))
locations = pd.read_csv(os.path.join(BASE_DIR, "raw_data", "Location Table.csv"))
```

Inspect

```python
trips.head(5)
```

	Trip ID	Pickup Time	Drop Off Time	passenger_count	trip_distance	PULocationID	DOLocationID	fare_amount	Surge Fee	Vehicle	Payment_type
0	1	6/1/2024 0:42	6/1/2024 1:04	1	5.60	79	226	19.5	2.0	UberX	Uber Pay
1	2	6/1/2024 0:06	6/1/2024 0:13	1	1.72	142	186	8.0	0.0	Uber Black	Cash
2	3	6/1/2024 0:08	6/1/2024 0:21	1	3.41	229	238	13.0	0.0	Uber Black	Cash
3	4	6/1/2024 0:28	6/1/2024 0:37	1	1.81	188	35	9.0	0.0	UberX	Cash
4	5	6/1/2024 0:38	6/1/2024 0:45	1	1.89	100	137	8.0	0.0	Uber Black	Cash

```python
trips.shape
```

```
(103728, 11)
```

```python
trips.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103728 entries, 0 to 103727
Data columns (total 11 columns):
 #   Column           Non-Null Count   Dtype  
---  ------           --------------   -----  
 0   Trip ID          103728 non-null  int64  
 1   Pickup Time      103728 non-null  object 
 2   Drop Off Time    103728 non-null  object 
 3   passenger_count  103728 non-null  int64  
 4   trip_distance    103728 non-null  float64
 5   PULocationID     103728 non-null  int64  
 6   DOLocationID     103728 non-null  int64  
 7   fare_amount      103728 non-null  float64
 8   Surge Fee        103728 non-null  float64
 9   Vehicle          103728 non-null  object 
 10  Payment_type     103728 non-null  object 
dtypes: float64(3), int64(4), object(4)
memory usage: 8.7+ MB
```

```python
locations.head(5)
```

	LocationID	Location	City
0	1	Newark Airport	Newark, New Jersey
1	2	Jamaica Bay	Queens
2	3	Allerton/Pelham Gardens	The Bronx
3	4	Alphabet City	Manhattan
4	5	Arden Heights	Staten Island

```python
locations.shape
```

```
(265, 3)
```

```python
locations.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 265 entries, 0 to 264
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype 
---  ------      --------------  ----- 
 0   LocationID  265 non-null    int64 
 1   Location    264 non-null    object
 2   City        263 non-null    object
dtypes: int64(1), object(2)
memory usage: 6.3+ KB
```

ETL: Cleaning & Transformation

Clean column names

```python
trips.columns = trips.columns.str.strip().str.lower().str.replace(" ", "_")
```

Convert datetime

```python
trips['pickup_time'] = pd.to_datetime(trips['pickup_time'])
trips['drop_off_time'] = pd.to_datetime(trips['drop_off_time'])
```

Feature engineering

```python
trips['trip_duration_minutes'] = (trips['drop_off_time'] - trips['pickup_time']).dt.total_seconds()/60
trips['total_fare'] = trips['fare_amount'] + trips['surge_fee']
trips['hour'] = trips['pickup_time'].dt.hour
trips['day'] = trips['pickup_time'].dt.day
trips['month'] = trips['pickup_time'].dt.month
trips['weekday'] = trips['pickup_time'].dt.day_name()
```

Clean location table

```python
locations.columns = locations.columns.str.strip().str.lower().str.replace(" ", "_")
```

Merge pickup & dropoff locations

```python
trips = trips.merge(locations, left_on="pulocationid", right_on="locationid", how="left", suffixes=("", "_pickup"))
trips = trips.merge(locations, left_on="dolocationid", right_on="locationid", how="left", suffixes=("", "_dropoff"))
```

```python
trips.head(5)
```

	trip_id	pickup_time	drop_off_time	passenger_count	trip_distance	pulocationid	dolocationid	fare_amount	surge_fee	vehicle	...	hour	day	month	weekday	locationid	location	city	locationid_dropoff	location_dropoff	city_dropoff
0	1	2024-06-01 00:42:00	2024-06-01 01:04:00	1	5.60	79	226	19.5	2.0	UberX	...	0	1	6	Saturday	79	East Village	Brooklyn	226	Sunnyside	Queens
1	2	2024-06-01 00:06:00	2024-06-01 00:13:00	1	1.72	142	186	8.0	0.0	Uber Black	...	0	1	6	Saturday	142	Lincoln Square East	Manhattan	186	Penn Station/Madison Sq West	Manhattan
2	3	2024-06-01 00:08:00	2024-06-01 00:21:00	1	3.41	229	238	13.0	0.0	Uber Black	...	0	1	6	Saturday	229	Sutton Place/Turtle Bay North	Manhattan	238	Upper West Side North	Manhattan
3	4	2024-06-01 00:28:00	2024-06-01 00:37:00	1	1.81	188	35	9.0	0.0	UberX	...	0	1	6	Saturday	188	Prospect-Lefferts Gardens	Brooklyn	35	Brownsville	Brooklyn
4	5	2024-06-01 00:38:00	2024-06-01 00:45:00	1	1.89	100	137	8.0	0.0	Uber Black	...	0	1	6	Saturday	100	Garment District	Queens	137	Kips Bay	Manhattan

5 rows × 23 columns

EDA: Exploratory Data Analysis

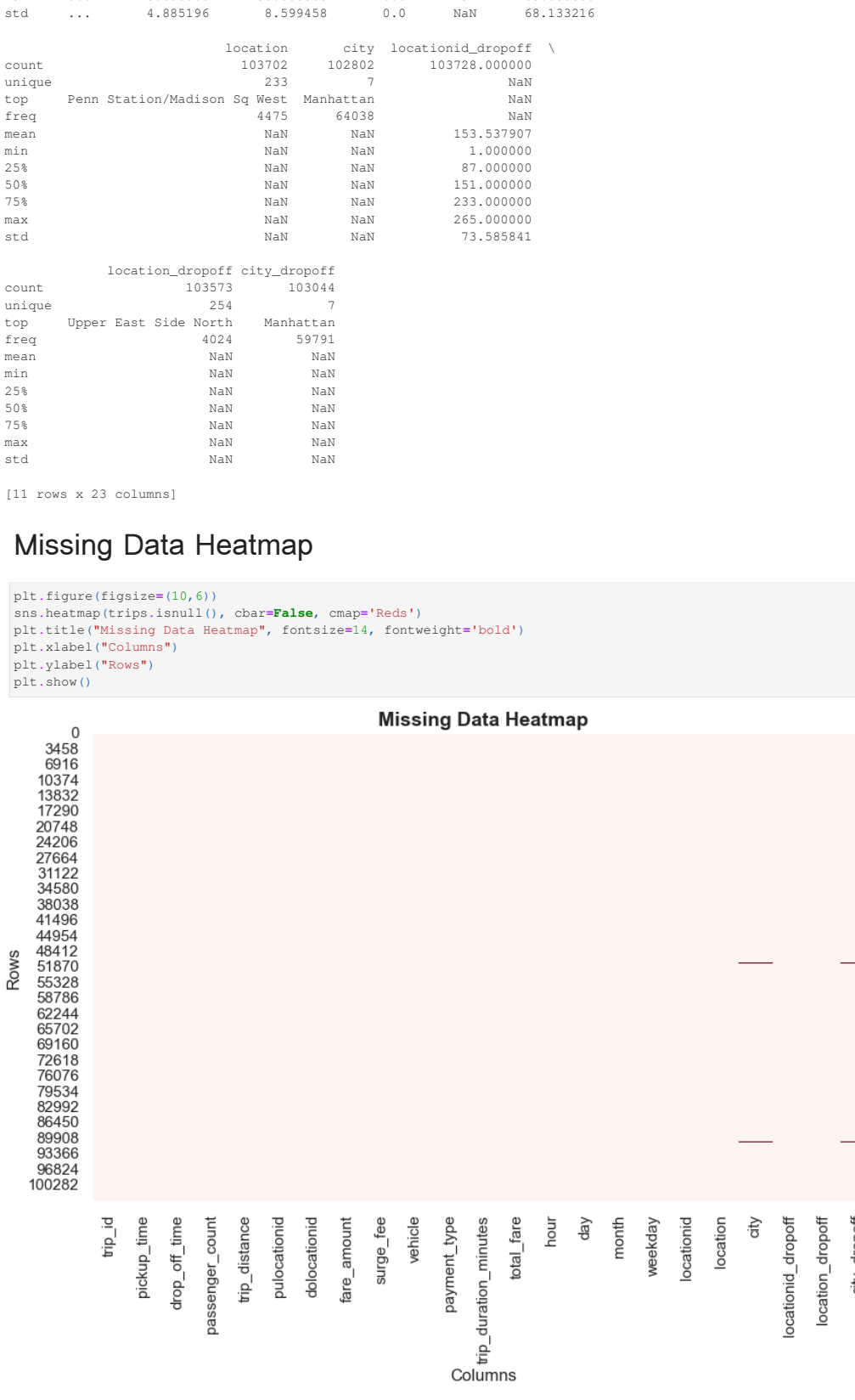
Dataset overview

```python
print("Missing values:\n", trips.isnull().sum())
print("Summary statistics:\n", trips.describe(include="all"))
```

```
Missing values:
 trip_id                  0
pickup_time              0
drop_off_time            0
passenger_count          0
trip_distance            0
pulocationid             0
dolocationid             0
fare_amount              0
surge_fee                0
vehicle                  0
payment_type             0
trip_duration_minutes    0
total_fare               0
hour                     0
day                      0
month                    0
weekday                  0
locationid               0
location                26
city                   926
locationid_dropoff       0
location_dropoff       155
city_dropoff           684
dtype: int64
Summary statistics:
               trip_id                    pickup_time  \
count   103728.000000                         103728   
unique            NaN                            NaN   
top               NaN                            NaN   
freq              NaN                            NaN   
mean     57422.229196  2024-06-17 08:18:07.280772864   
min          1.000000            2024-06-01 00:00:00   
25%      27899.750000            2024-06-10 07:09:00   
50%      56618.500000            2024-06-17 19:37:00   
75%      86859.250000            2024-06-24 18:45:15   
max     116924.000000            2024-06-30 23:58:00   
std      33812.530174                            NaN   

                        drop_off_time  passenger_count  trip_distance  \
count                          103728    103728.000000  103728.000000   
unique                            NaN              NaN            NaN   
top                               NaN              NaN            NaN   
freq                              NaN              NaN            NaN   
mean    2024-06-17 08:33:59.035169024         1.412136       3.363931   
min               2024-06-01 00:10:00         1.000000       1.010000   
25%               2024-06-10 07:21:45         1.000000       1.530000   
50%               2024-06-17 19:50:00         1.000000       2.280000   
75%               2024-06-24 18:58:00         1.000000       3.800000   
max               2024-07-01 17:21:00         6.000000     144.100000   
std                               NaN         1.031418       3.192828   

         pulocationid   dolocationid    fare_amount      surge_fee vehicle  \
count   103728.000000  103728.000000  103728.000000  103728.000000  103728   
unique            NaN            NaN            NaN            NaN       5   
top               NaN            NaN            NaN            NaN   UberX   
freq              NaN            NaN            NaN            NaN   38744   
mean       159.241410     153.537907      12.998337       1.979999     NaN   
min          1.000000       1.000000       2.500000       0.000000     NaN   
25%        107.000000      87.000000       7.500000       0.000000     NaN   
50%        151.000000     151.000000      10.000000       1.960000     NaN   
75%        234.000000     233.000000      14.500000       3.000000     NaN   
max        265.000000     265.000000     563.790000     117.930000     NaN   
std         68.133216      73.585841       9.078582       2.421460     NaN   

        ...           hour            day     month  weekday     locationid  \
count   ...  103728.000000  103728.000000  103728.0   103728  103728.000000   
unique  ...            NaN            NaN       NaN        7            NaN   
top     ...            NaN            NaN       NaN   Sunday            NaN   
freq    ...            NaN            NaN       NaN    19165            NaN   
mean    ...      13.600648      16.758580       6.0      NaN     159.241410   
min     ...       0.000000       1.000000       6.0      NaN       1.000000   
25%     ...      10.000000      10.000000       6.0      NaN     107.000000   
50%     ...      14.000000      17.000000       6.0      NaN     151.000000   
75%     ...      17.000000      24.000000       6.0      NaN     234.000000   
max     ...      23.000000      30.000000       6.0      NaN     265.000000   
std     ...       4.885196       8.599458       0.0      NaN      68.133216   

                           location       city  locationid_dropoff  \
count                        103702     102802       103728.000000   
unique                          233          7                 NaN   
top     Penn Station/Madison Sq West  Manhattan                 NaN   
freq                           4475      64038                 NaN   
mean                            NaN        NaN          153.537907   
min                             NaN        NaN            1.000000   
25%                             NaN        NaN           87.000000   
50%                             NaN        NaN          151.000000   
75%                             NaN        NaN          233.000000   
max                             NaN        NaN          265.000000   
std                             NaN        NaN           73.585841   

             location_dropoff city_dropoff  
count                  103573       103044  
unique                    254            7  
top     Upper East Side North    Manhattan  
freq                     4024        59791  
mean                      NaN          NaN  
min                       NaN          NaN  
25%                       NaN          NaN  
50%                       NaN          NaN  
75%                       NaN          NaN  
max                       NaN          NaN  
std                       NaN          NaN  

[11 rows x 23 columns]
```

Missing Data Heatmap

```python
plt.figure(figsize=(10,6))
sns.heatmap(trips.isnull(), cbar=False, cmap='Reds')
plt.title("Missing Data Heatmap", fontsize=14, fontweight='bold')
plt.xlabel("Columns")
plt.ylabel("Rows")
plt.show()
```



Missing Values handling

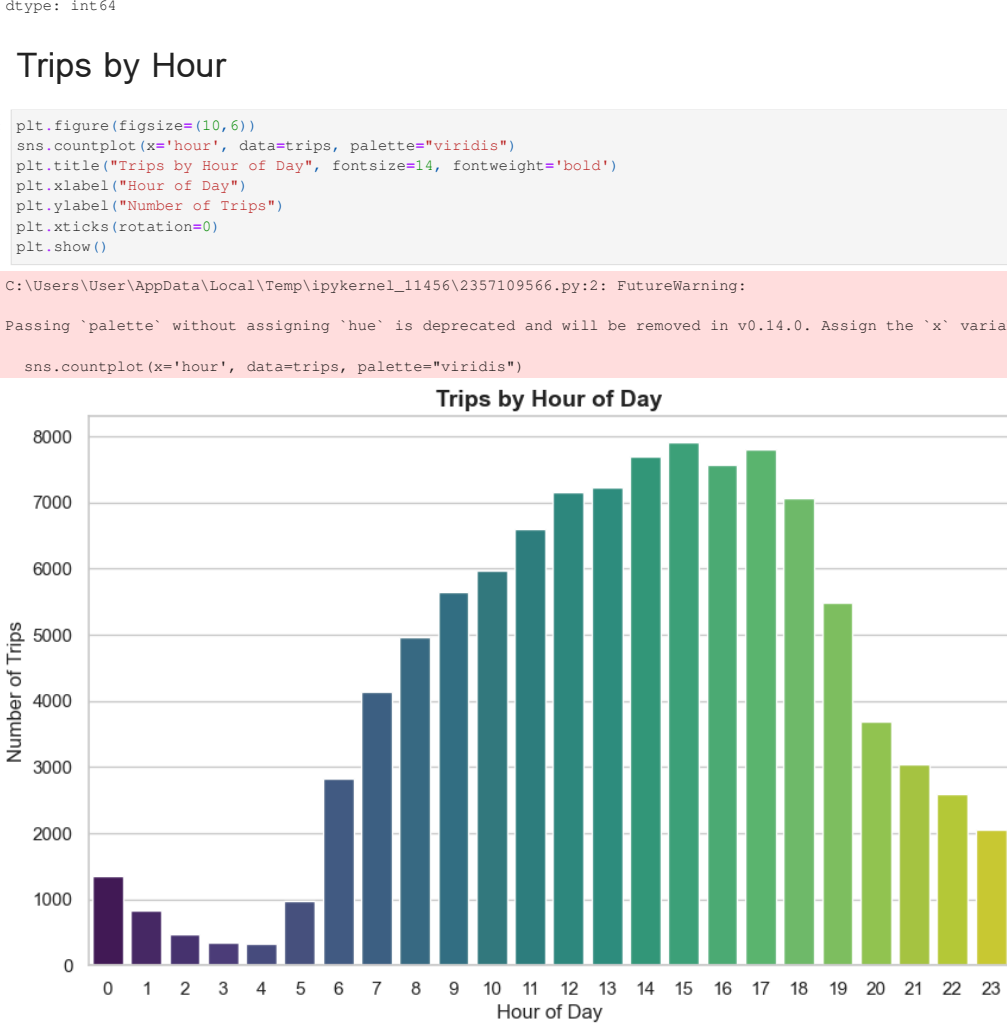
```python
trips['location'] = trips['location'].fillna('Unknown')
trips['city'] = trips['city'].fillna('Unknown')
trips['location_dropoff'] = trips['location_dropoff'].fillna('Unknown')
trips['city_dropoff'] = trips['city_dropoff'].fillna('Unknown')

print("Missing values after cleanup:\n", trips[['location','city','location_dropoff','city_dropoff']].isnull().sum())
```

```
Missing values after cleanup:
 location            0
city                0
location_dropoff    0
city_dropoff        0
dtype: int64
```

Trips by Hour

```python
plt.figure(figsize=(10,6))
sns.countplot(x='hour', data=trips, palette="viridis")
plt.title("Trips by Hour of Day", fontsize=14, fontweight='bold')
plt.xlabel("Hour of Day")
plt.ylabel("Number of Trips")
plt.xticks(rotation=0)
plt.show()
```

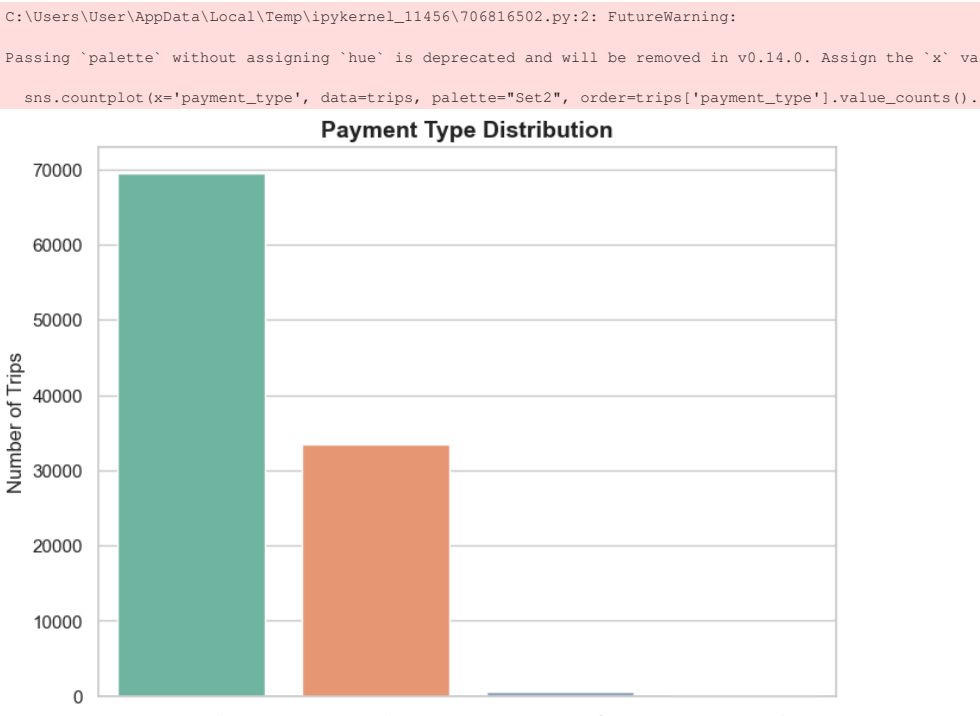
```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\2357109566.py:2: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='hour', data=trips, palette="viridis")
```



Revenue by Vehicle Type

```python
plt.figure(figsize=(10,6))
revenue_by_vehicle = trips.groupby('vehicle')['total_fare'].sum().sort_values(ascending=False)
sns.barplot(x=revenue_by_vehicle.index, y=revenue_by_vehicle.values, palette="Blues_d")
plt.title("Revenue by Vehicle Type", fontsize=14, fontweight='bold')
plt.xlabel("Vehicle Type")
plt.ylabel("Total Revenue ($)")
plt.xticks(rotation=45)
plt.show()
```

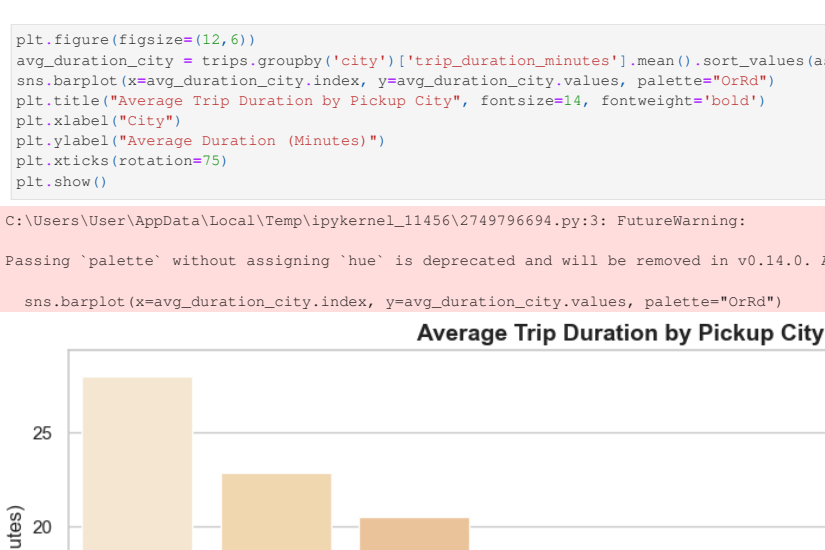
```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\2304027568.py:3: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=revenue_by_vehicle.index, y=revenue_by_vehicle.values, palette="Blues_d")
```



Payment Type Distribution

```python
plt.figure(figsize=(8,6))
sns.countplot(x='payment_type', data=trips, palette="Set2", order=trips['payment_type'].value_counts().index)
plt.title("Payment Type Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Payment Method")
plt.ylabel("Number of Trips")
plt.xticks(rotation=30)
plt.show()
```

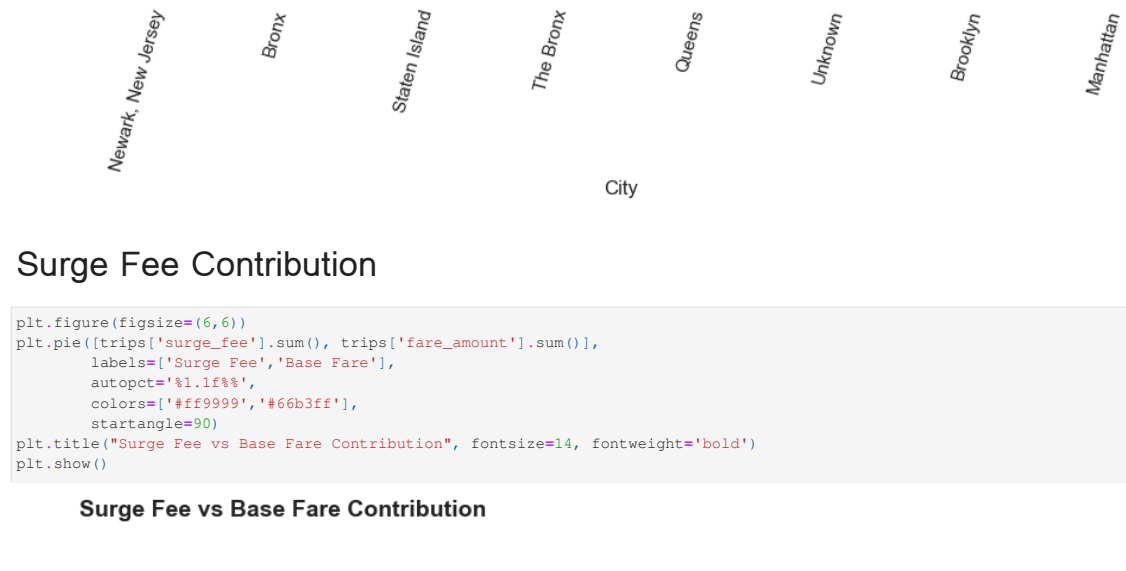
```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\706816502.py:2: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='payment_type', data=trips, palette="Set2", order=trips['payment_type'].value_counts().index)
```



Average Trip Duration by Pickup City

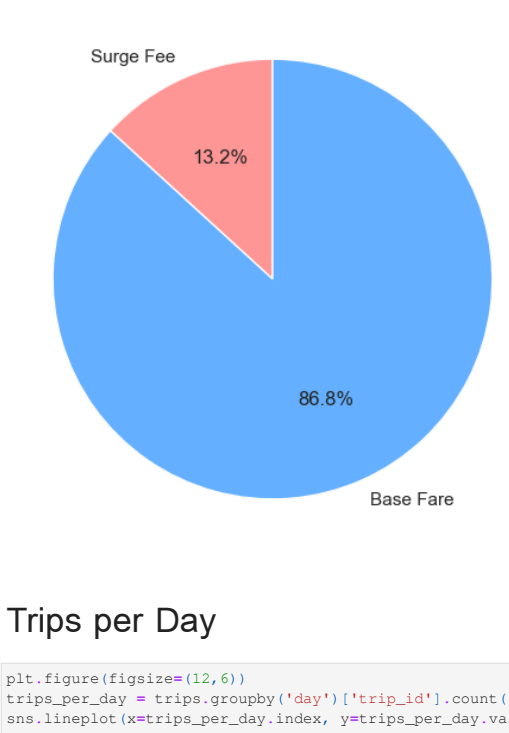
```python
plt.figure(figsize=(12,6))
avg_duration_city = trips.groupby('city')['trip_duration_minutes'].mean().sort_values(ascending=False)
sns.barplot(x=avg_duration_city.index, y=avg_duration_city.values, palette="OrRd")
plt.title("Average Trip Duration by Pickup City", fontsize=14, fontweight='bold')
plt.xlabel("City")
plt.ylabel("Average Duration (Minutes)")
plt.xticks(rotation=75)
plt.show()
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\2749796694.py:3: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=avg_duration_city.index, y=avg_duration_city.values, palette="OrRd")
```



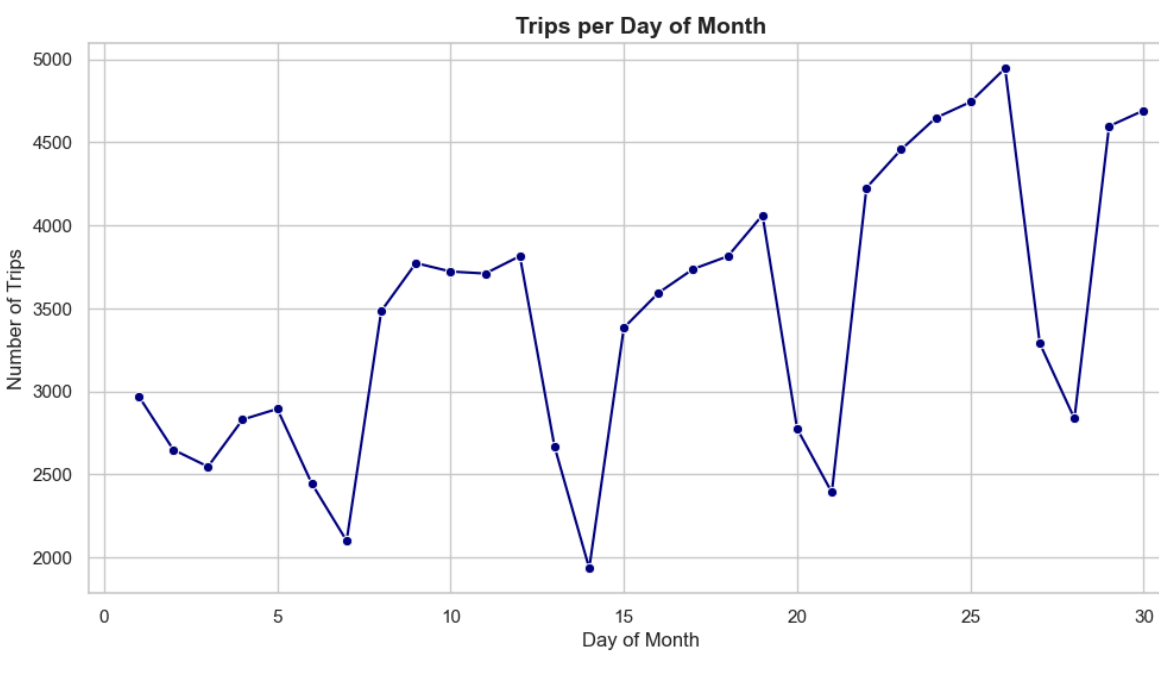
Surge Fee Contribution

```python
plt.figure(figsize=(6,6))
plt.pie([trips['surge_fee'].sum(), trips['fare_amount'].sum()],
        labels=['Surge Fee','Base Fare'],
        autopct='%1.1f%%',
        colors=['#ff9999','#66b3ff'],
        startangle=90)
plt.title("Surge Fee vs Base Fare Contribution", fontsize=14, fontweight='bold')
plt.show()
```



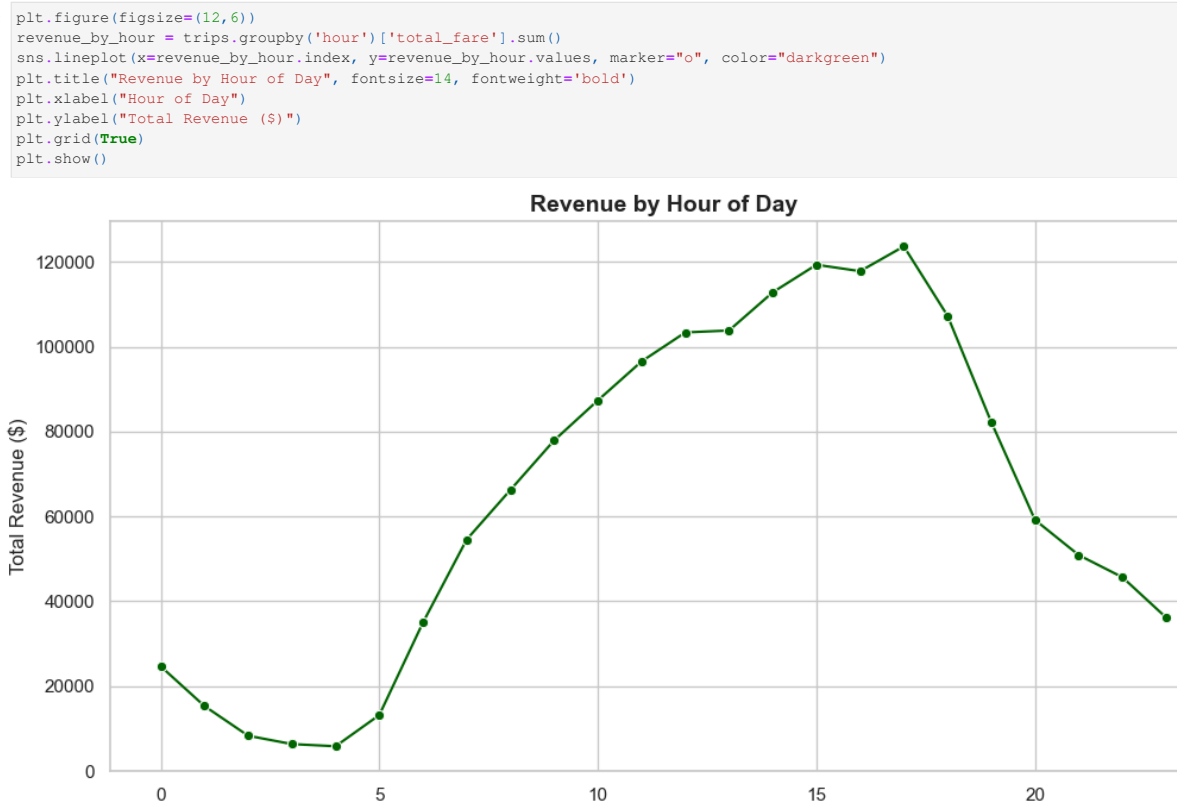
Trips per Day

```python
plt.figure(figsize=(12,6))
trips_per_day = trips.groupby('day')['trip_id'].count()
sns.lineplot(x=trips_per_day.index, y=trips_per_day.values, marker="o", color="navy")
plt.title("Trips per Day of Month", fontsize=14, fontweight='bold')
plt.xlabel("Day of Month")
plt.ylabel("Number of Trips")
plt.grid(True)
plt.show()
```



Revenue Trend by Hour

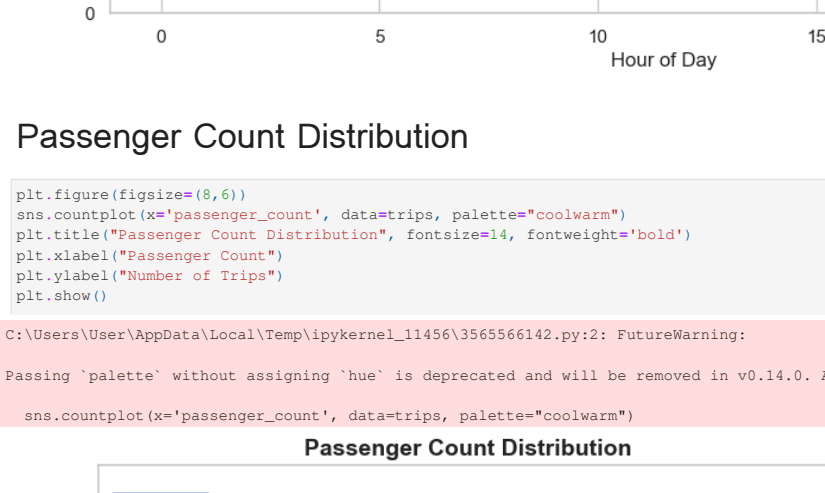
```python
plt.figure(figsize=(12,6))
revenue_by_hour = trips.groupby('hour')['total_fare'].sum()
sns.lineplot(x=revenue_by_hour.index, y=revenue_by_hour.values, marker="o", color="darkgreen")
plt.title("Revenue by Hour of Day", fontsize=14, fontweight='bold')
plt.xlabel("Hour of Day")
plt.ylabel("Total Revenue ($)")
plt.grid(True)
plt.show()
```



Passenger Count Distribution

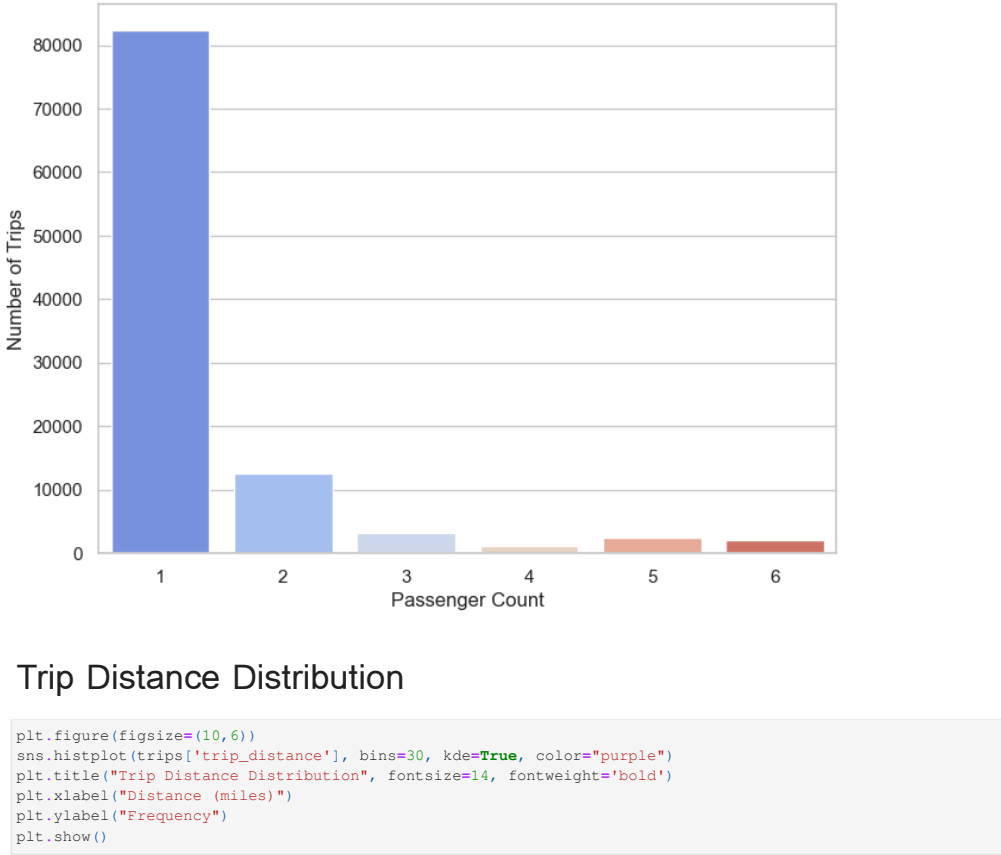
```python
plt.figure(figsize=(8,6))
sns.countplot(x='passenger_count', data=trips, palette="coolwarm")
plt.title("Passenger Count Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Passenger Count")
plt.ylabel("Number of Trips")
plt.show()
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\3565566142.py:2: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.countplot(x='passenger_count', data=trips, palette="coolwarm")
```



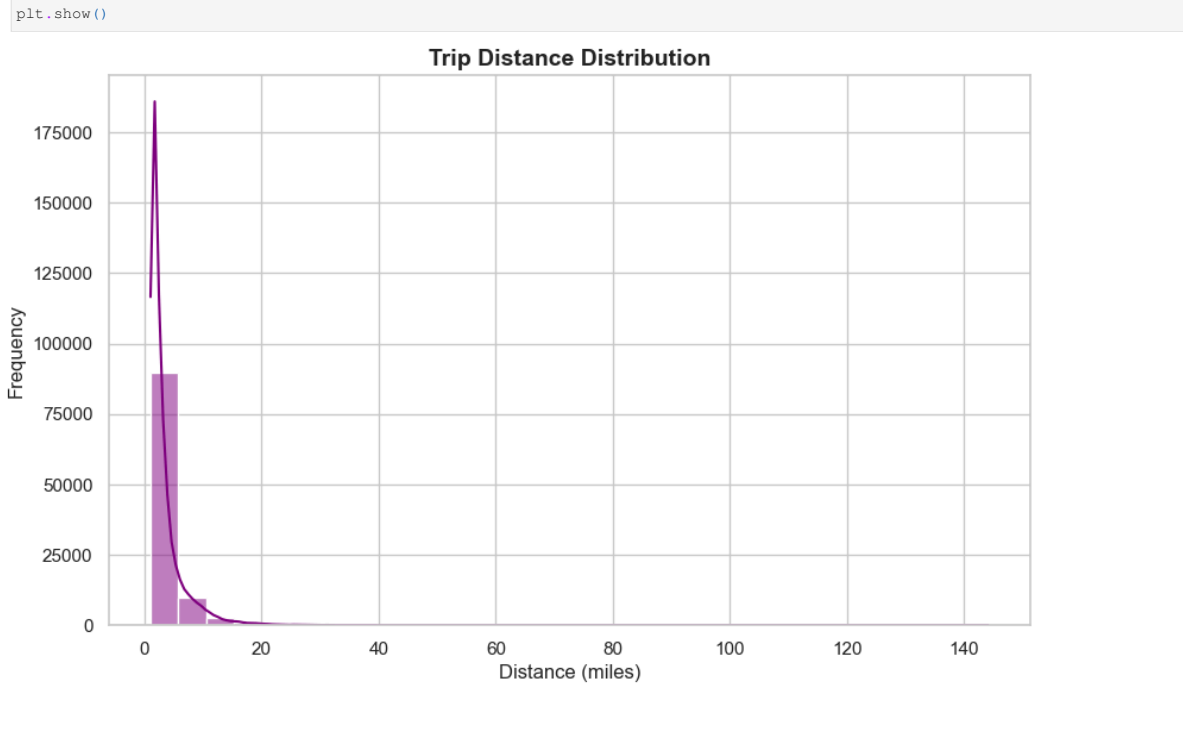
Trip Distance Distribution

```python
plt.figure(figsize=(10,6))
sns.histplot(trips['trip_distance'], bins=30, kde=True, color="purple")
plt.title("Trip Distance Distribution", fontsize=14, fontweight='bold')
plt.xlabel("Distance (miles)")
plt.ylabel("Frequency")
plt.show()
```



Fare vs Distance Scatter

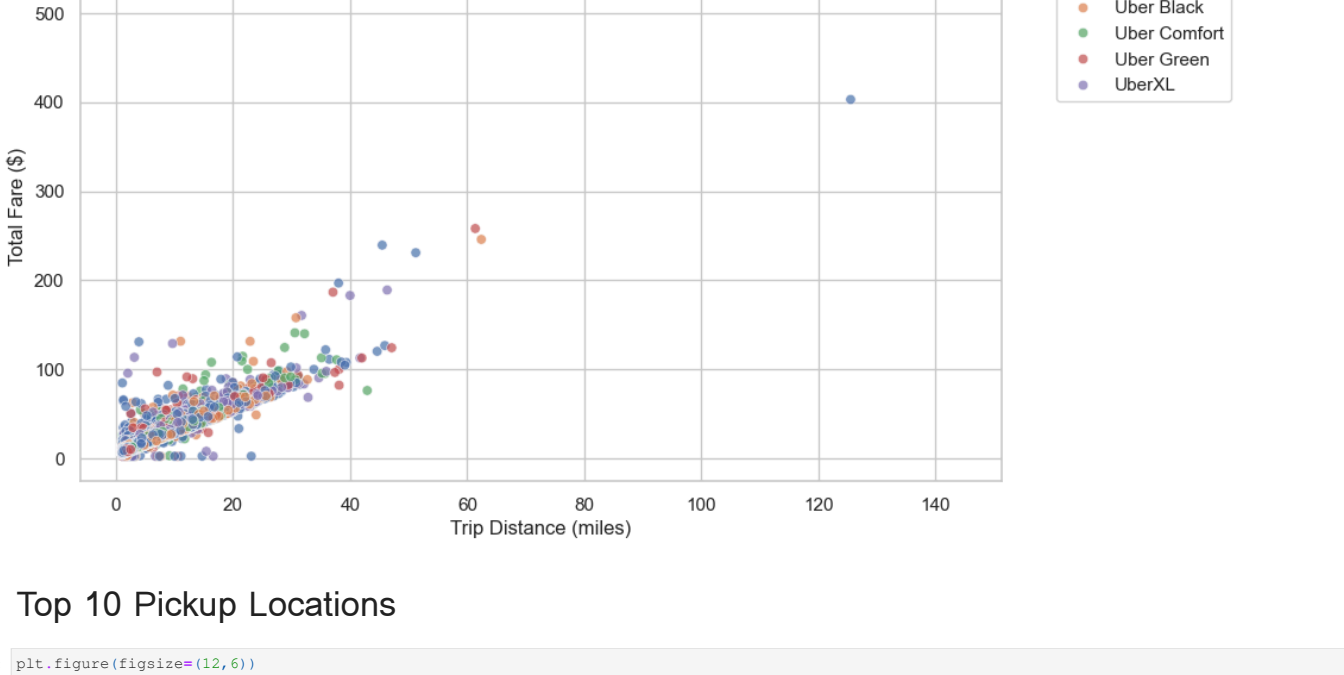
```python
plt.figure(figsize=(10,6))
sns.scatterplot(x='trip_distance', y='total_fare', data=trips, hue='vehicle', alpha=0.7)
plt.title("Fare vs Trip Distance by Vehicle Type", fontsize=14, fontweight='bold')
plt.xlabel("Trip Distance (miles)")
plt.ylabel("Total Fare ($)")
plt.legend(title="Vehicle Type", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.show()
```



Top 10 Pickup Locations

```python
plt.figure(figsize=(12,6))

top_pickups = trips['location'].value_counts().head(10)

sns.barplot(x=top_pickups.index, y=top_pickups.values, palette="crest")

plt.title("Top 10 Pickup Locations", fontsize=16, fontweight='bold')
plt.xlabel("Pickup Location", fontsize=12)
plt.ylabel("Number of Trips", fontsize=12)
plt.xticks(rotation=75, ha="right")
plt.grid(axis='y', linestyle='--', alpha=0.7)

for i, v in enumerate(top_pickups.values):
    plt.text(i, v + 0.5, str(v), ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```

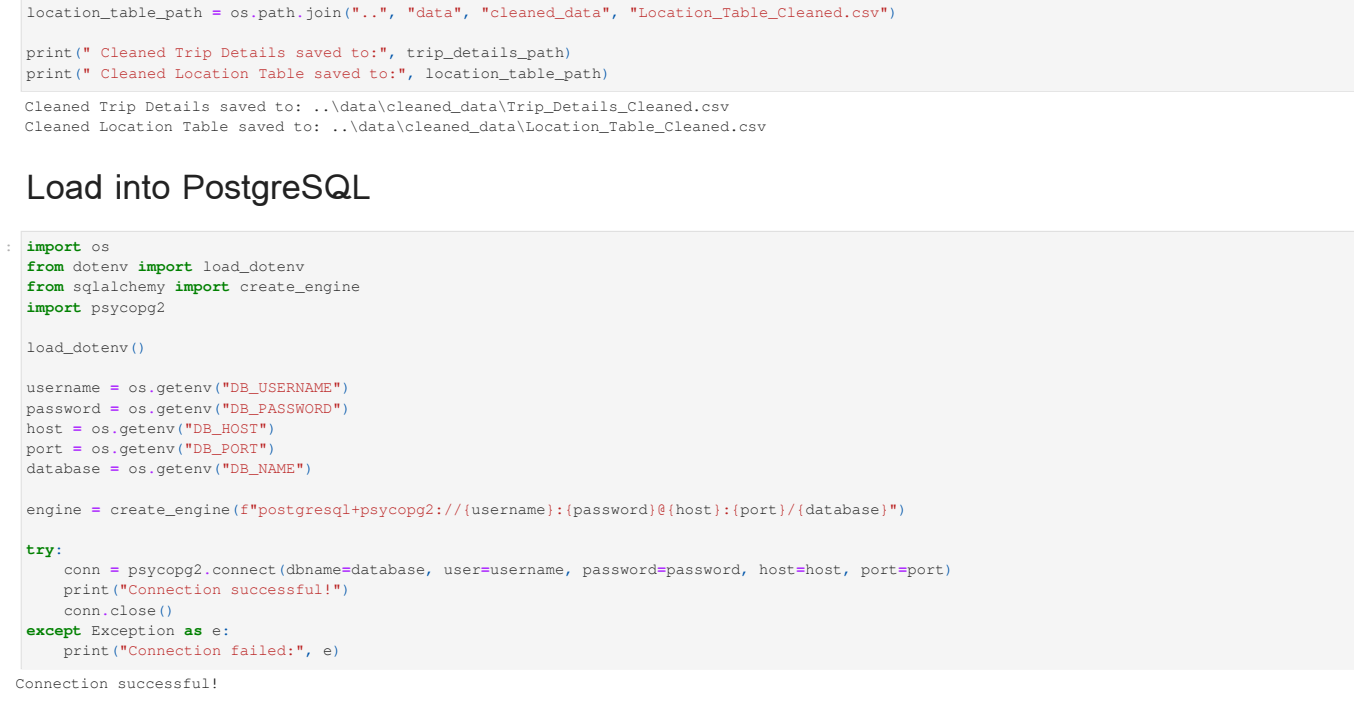
```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\187316281.py:5: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=top_pickups.index, y=top_pickups.values, palette="crest")
```



Top 10 Dropoff Locations

```python
plt.figure(figsize=(12,6))

top_dropoffs = trips['location_dropoff'].value_counts().head(10)

sns.barplot(x=top_dropoffs.index, y=top_dropoffs.values, palette="mako")

plt.title("Top 10 Dropoff Locations", fontsize=16, fontweight='bold')
plt.xlabel("Dropoff Location", fontsize=12)
plt.ylabel("Number of Trips", fontsize=12)
plt.xticks(rotation=75, ha="right")
plt.grid(axis='y', linestyle='--', alpha=0.7)

for i, v in enumerate(top_dropoffs.values):
    plt.text(i, v + 0.5, str(v), ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_11456\2840604226.py:5: FutureWarning: 

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.barplot(x=top_dropoffs.index, y=top_dropoffs.values, palette="mako")
```



Save Cleaned Data Locally

Save to CSV

```python
trip_details_path = os.path.join("..", "data", "cleaned_data", "Trip_Details_Cleaned.csv")
location_table_path = os.path.join("..", "data", "cleaned_data", "Location_Table_Cleaned.csv")

print(" Cleaned Trip Details saved to:", trip_details_path)
print(" Cleaned Location Table saved to:", location_table_path)
```

```
Cleaned Trip Details saved to: ..\data\cleaned_data\Trip_Details_Cleaned.csv
Cleaned Location Table saved to: ..\data\cleaned_data\Location_Table_Cleaned.csv
```

Load into PostgreSQL

```python
import os
from dotenv import load_dotenv
from sqlalchemy import create_engine
import psycopg2

load_dotenv()

username = os.getenv("DB_USERNAME")
password = os.getenv("DB_PASSWORD")
host = os.getenv("DB_HOST")
port = os.getenv("DB_PORT")
database = os.getenv("DB_NAME")

engine = create_engine(f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

try:
    conn = psycopg2.connect(dbname=database, user=username, password=password, host=host, port=port)
    print("Connection successful!")
    conn.close()
except Exception as e:
    print("Connection failed:", e)
```

```
Connection successful!
```

Upload tables

```python
trips.to_sql("Trip_Details", engine, if_exists="replace", index=False)
locations.to_sql("Location_Table", engine, if_exists="replace", index=False)
```

```
265
```

Business-Oriented ML Models

Fare Prediction Model (Regression)

Goal: Predict trip fare based on distance, duration, surge, and vehicle type.

Business Impact: Helps optimize dynamic pricing strategies and anticipate revenue.

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np


X = trips[['trip_distance', 'trip_duration_minutes', 'passenger_count']]
y = trips['fare_amount']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

fare_model = RandomForestRegressor(n_estimators=200, random_state=42)
fare_model.fit(X_train, y_train)

y_pred = fare_model.predict(X_test)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("Fare Prediction RMSE:", rmse)
print("Fare Prediction R²:", r2)
```

```
Fare Prediction RMSE: 0.9123285456983095
Fare Prediction R²: 0.9894154762216791
```

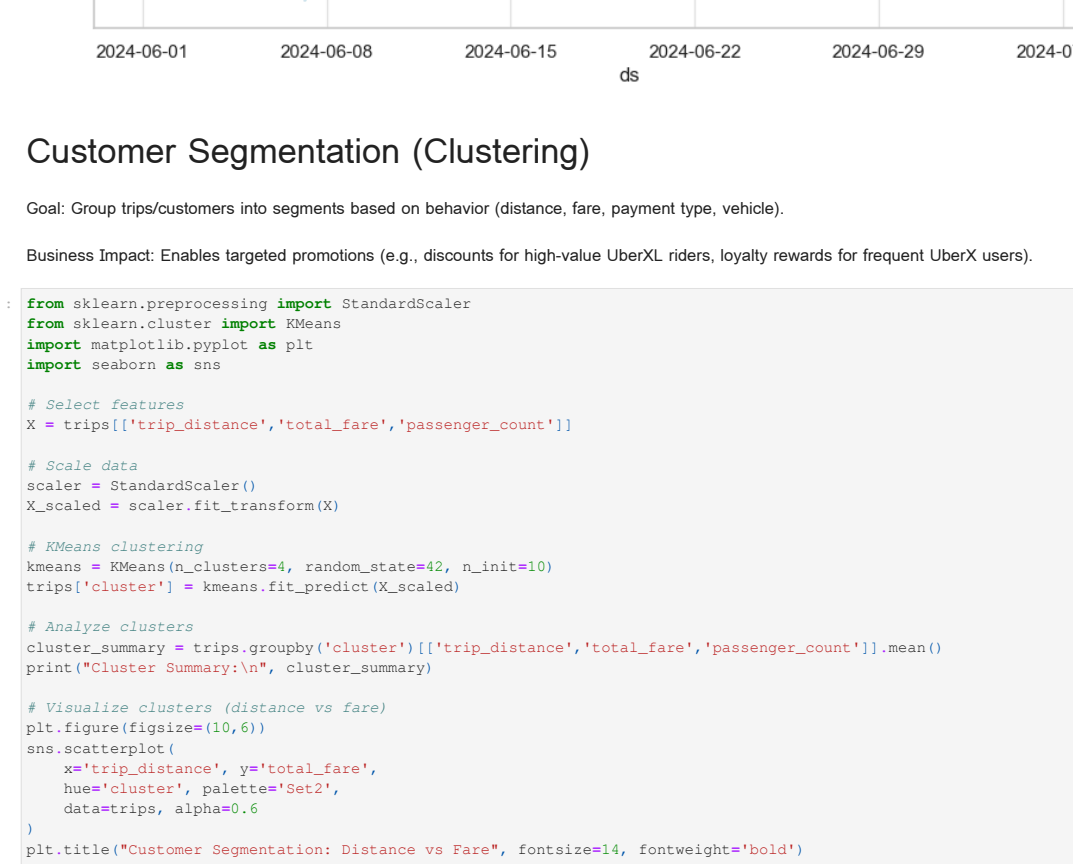
Demand Forecasting Model (Time Series)

Goal: Forecast trip demand by hour/day.

Business Impact: Helps allocate drivers efficiently, reduce wait times, and maximize utilization.

```python
from prophet import Prophet

# Prepare time series data
demand = trips.groupby('pickup_time').size().reset_index(name='trips')
demand.rename(columns={'pickup_time':'ds','trips':'y'}, inplace=True)

# Train Prophet model
model = Prophet()
model.fit(demand)

# Forecast next 7 days
future = model.make_future_dataframe(periods=168, freq='h')  # 168 hours = 7 days
forecast = model.predict(future)

# Plot forecast
model.plot(forecast)
plt.title("Forecasted Trip Demand (Next 7 Days)", fontsize=14, fontweight='bold')
plt.show()
```

```
00:43:10 - cmdstanpy - INFO - Chain [1] start processing
00:43:13 - cmdstanpy - INFO - Chain [1] done processing
```



Customer Segmentation (Clustering)

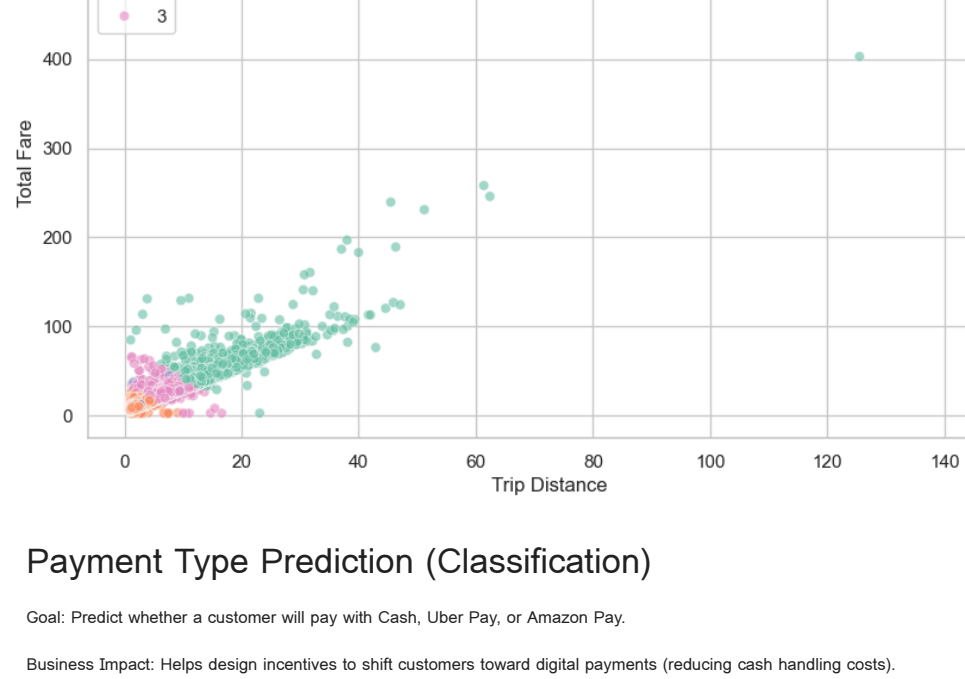
Goal: Group trips/customers into segments based on behavior (distance, fare, payment type, vehicle).

Business Impact: Enables targeted promotions (e.g., discounts for high-value UberXL riders, loyalty rewards for frequent UberX users).

```python
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import seaborn as sns

# Select features
X = trips[['trip_distance','total_fare','passenger_count']]

# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# KMeans clustering
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
trips['cluster'] = kmeans.fit_predict(X_scaled)

# Analyze clusters
cluster_summary = trips.groupby('cluster')[['trip_distance','total_fare','passenger_count']].mean()
print("Cluster Summary:\n", cluster_summary)

# Visualize clusters (distance vs fare)
plt.figure(figsize=(10,6))
sns.scatterplot(
    x='trip_distance', y='total_fare',
    hue='cluster', palette='Set2',
    data=trips, alpha=0.6
)
plt.title("Customer Segmentation: Distance vs Fare", fontsize=14, fontweight='bold')
plt.xlabel("Trip Distance")
plt.ylabel("Total Fare")
plt.legend(title="Cluster")
plt.show()
```

```
c:\Users\User\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py:136: UserWarning: Could not find the number of physical cores for the following reason:
[WinError 2] The system cannot find the file specified
Returning the number of logical cores instead. You can silence this warning by setting LOKY_MAX_CPU_COUNT to the number of cores you want to use.
  warnings.warn(
  File "c:\Users\User\anaconda3\Lib\site-packages\joblib\externals\loky\backend\context.py", line 257, in _count_physical_cores
    cpu_info = subprocess.run(
        "wmic CPU Get NumberOfCores /Format:csv".split(),
        capture_output=True,
        text=True,
    )
  File "c:\Users\User\anaconda3\Lib\subprocess.py", line 554, in run
    with Popen(*popenargs, **kwargs) as process:
         ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\User\anaconda3\Lib\subprocess.py", line 1039, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
    ~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                        pass_fds, cwd, env,
                        ^^^^^^^^^^^^^^^^^^^
    ...<5 lines>...
                        gid, gids, uid, umask,
                        ^^^^^^^^^^^^^^^^^^^^^^
                        start_new_session, process_group)
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\User\anaconda3\Lib\subprocess.py", line 1554, in _execute_child
    hp, ht, pid, tid = _winapi.CreateProcess(executable, args,
                       ~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
                             # no special security
                             ^^^^^^^^^^^^^^^^^^^^^
    ...<4 lines>...
                             cwd,
                             ^^^^
                             startupinfo)
                             ^^^^^^^^^^^^
```

```
Cluster Summary:
          trip_distance  total_fare  passenger_count
cluster                                             
0            15.354208   50.932120         1.397915
1             2.147419   11.129222         1.129810
2             2.788428   13.204244         4.465494
3             6.731225   26.026217         1.181957
```



Payment Type Prediction (Classification)

Goal: Predict whether a customer will pay with Cash, Uber Pay, or Amazon Pay.

Business Impact: Helps design incentives to shift customers toward digital payments (reducing cash handling costs).

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
trips['payment_encoded'] = le.fit_transform(trips['payment_type'])

X = trips[['trip_distance','trip_duration_minutes','total_fare','surge_fee']]
y = trips['payment_encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)


clf = RandomForestClassifier(n_estimators=200, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

```
              precision    recall  f1-score   support

  Amazon Pay       0.02      0.13      0.04       117
        Cash       0.89      0.85      0.87      6687
  Google Pay       0.01      0.11      0.02        36
    Uber Pay       0.99      0.95      0.97     13906

    accuracy                           0.91     20746
   macro avg       0.48      0.51      0.48     20746
weighted avg       0.95      0.91      0.93     20746
```